A Low-Latency MQTT Bridge for Edge-Deployed Industrial Sensors

Abstract

Industrial IoT deployments increasingly mix legacy Modbus RTU sensors with modern MQTT-based telemetry platforms. Off-the-shelf bridges introduce latencies in the 80–200 ms range that are unacceptable for closed-loop control applications. We present a lightweight, single-binary MQTT bridge written in Rust that preserves sub-10 ms median latency on commodity edge hardware (Raspberry Pi 4) while keeping memory under 20 MB. The bridge is deployed at three manufacturing sites and has processed 1.4 billion messages without message loss across a six-month window.

I. INTRODUCTION

Industrial sensor networks deployed before 2015 predominantly use Modbus RTU over RS-485 serial links. Retrofitting these networks to feed modern cloud dashboards typically requires a protocol bridge. Existing open-source bridges such as modbus2mqtt and industrial-gateway incur JSON serialisation costs and Python GIL contention that push end-to-end latency well above 100 ms under moderate load. For process-control applications with tight control loops, this latency is prohibitive.

II. RELATED WORK

Boyer (2016) surveys the industrial protocol landscape and identifies the latency wall at roughly 50 ms for closed-loop control. Liu et al. (2020) propose a C++ bridge achieving 20 ms median latency but at the cost of a 200 MB memory footprint, unsuitable for many edge deployments. Our work differs from Liu et al. by targeting both sub-10 ms latency and sub-20 MB memory using Rust's zero-cost abstractions.

III. SYSTEM DESIGN

The bridge consists of three asynchronous tasks running on Tokio: (1) a Modbus poller that reads configured registers at a fixed interval, (2) an MQTT publisher that batches messages up to 16 at a time, and (3) a watchdog task that monitors the health of the other two and restarts them on failure.

We use a bounded lock-free channel (crossbeam_channel) between the poller and publisher to decouple polling jitter from publishing. Messages are serialised as MessagePack rather than JSON, which reduces payload size by roughly 40 percent and eliminates UTF-8 validation overhead.

IV. EXPERIMENTS

We benchmarked the bridge on a Raspberry Pi 4 (4 GB RAM, Raspberry Pi OS 64-bit) against modbus2mqtt and industrial-gateway under three load profiles: 10 sensors at 1 Hz, 50 sensors at 1 Hz, and 10 sensors at 100 Hz. Table 1 summarises median and 99th-percentile latency.

Under the 10 × 1 Hz profile our bridge achieves 4.2 ms median and 8.1 ms p99. Under 50 × 1 Hz the figures rise to 5.8 ms median and 14.2 ms p99. At 10 × 100 Hz they rise to 9.4 ms median and 22.1 ms p99. All three profiles are well under the 50 ms control-loop threshold identified by Boyer.

V. DEPLOYMENT EXPERIENCE

The bridge has been deployed at three sites since October 2025: a wastewater treatment plant in Pittsburgh, a tire factory in Akron, and a food-packaging line in Cincinnati. Across six months of operation it processed 1.42 billion messages with zero reported message losses and two watchdog-triggered restarts both caused by upstream MQTT broker outages. Memory usage stayed between 14 and 18 MB across all three sites.

VI. CONCLUSION

A Rust-native MQTT bridge with careful async design can bring Modbus sensor latencies well within the closed-loop control window on commodity edge hardware. We open-source the code under the MIT license and include deployment recipes for systemd, Balena, and K3s.

VII. REFERENCES

[1] S. Boyer, *SCADA: Supervisory Control and Data Acquisition*, 4th ed., ISA, 2016. [2] J. Liu, M. Park, and D. Chen, "A low-latency industrial protocol bridge," in *Proc. IEEE Int. Conf. Industrial Informatics*, 2020, pp. 412–418. DOI: 10.1109/INDIN48963.2020.9442123. [3] OASIS, *MQTT Version 5.0 Specification*, 2019. [4] Modbus Organization, *Modbus Application Protocol Specification V1.1b3*, 2012. [5] R. Klabnik and C. Nichols, *The Rust Programming Language*, No Starch Press, 2019. [6] Tokio Contributors, "Tokio: An asynchronous runtime for Rust," 2024. [7] MessagePack Contributors, "MessagePack specification," 2024. [8] E. Kohler and R. Morris, "The Click modular router," *ACM

Trans. Comput. Syst.*, vol. 18, no. 3, 2000. DOI: 10.1145/354871.354874. [9] IEEE Standard 802.1Q-2022, "Bridges and Bridged Networks," 2022. [10] Raspberry Pi Foundation, "Raspberry Pi 4 Model B Product Brief," 2019. [11] OpenSSL Project, "OpenSSL 3.0 Release Notes," 2021. [12] D. Thaler and B. Aboba, "What Makes for a Successful Protocol?", RFC 5218, 2008.